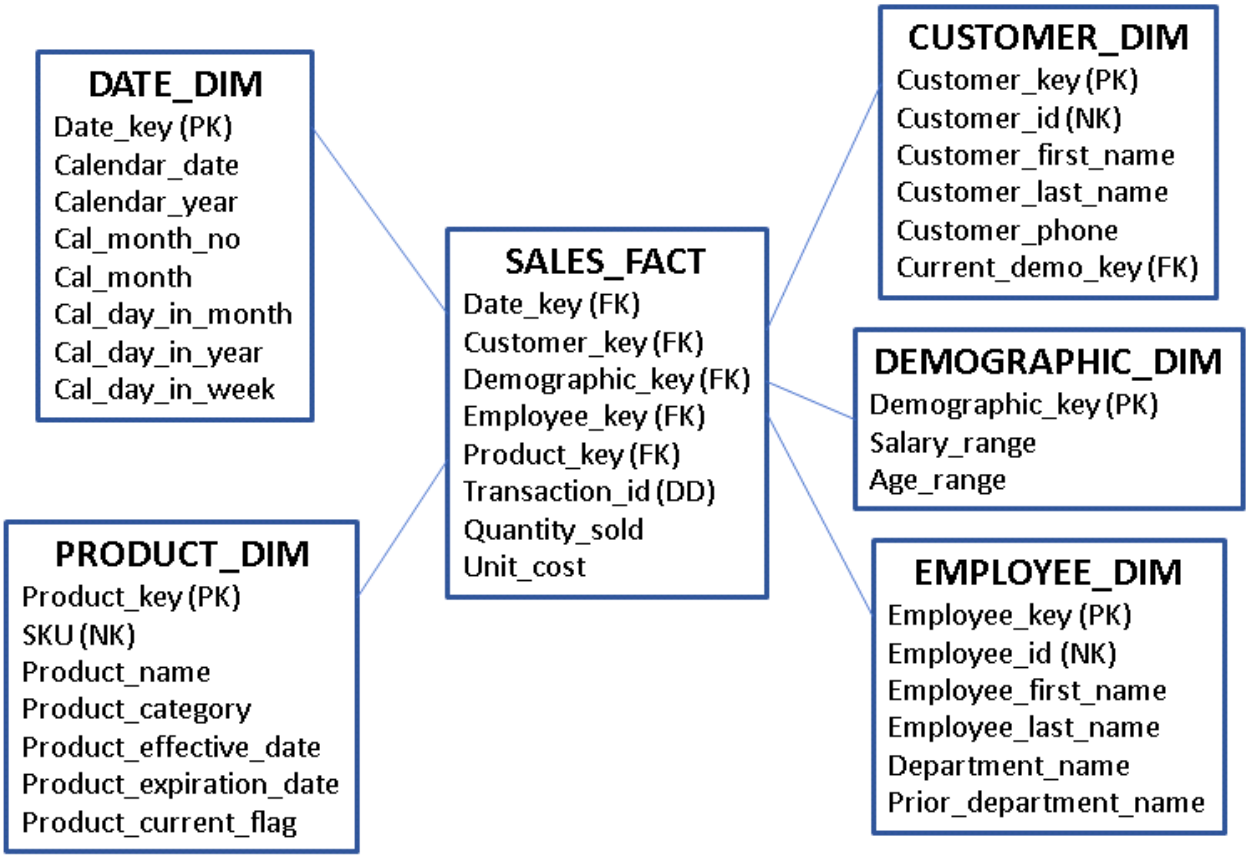
# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

**Lab Description:** This lab utilizes a simple toy store retail sales dimensional data model, found in the class database, in the schema SAMPLES. This data model incorporates Slowly Changing Dimension (SCL) Types 0-5. Follow the lab descriptions and SQL examples to experience the SCL types hands-on.

See document **"Connecting to Class Database via SQL Developer.pdf"** found under the Project Files folder of the class Canvas site to get connected to the class database.

## Toy Store Retail Sales Dimensional Data Model

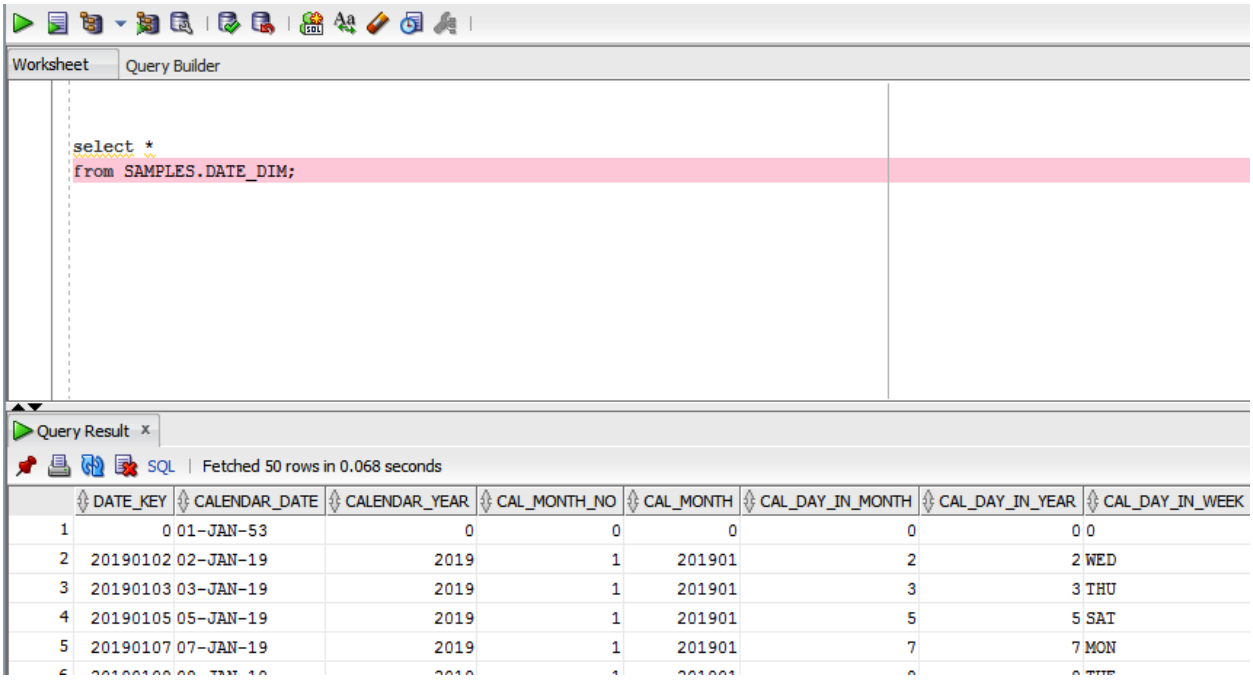


# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

**TYPE 0: Never changes:**

Most attributes in a date dimension would qualify as type 0 SCD, because they will never change once entered, so no need to track historical change. Run a simple query against the date dimension to view the available data.



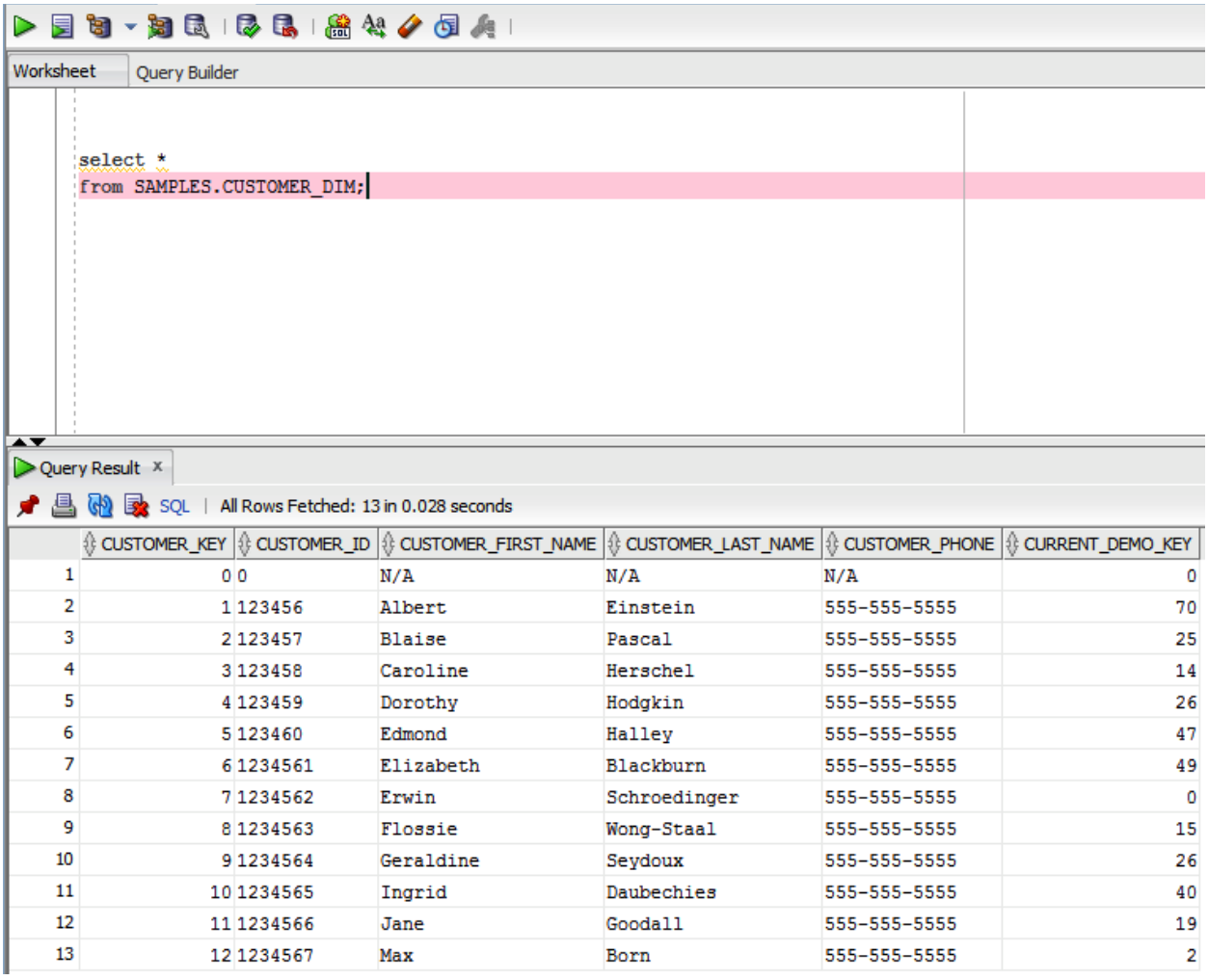

| | DATE_KEY | CALENDAR_DATE | CALENDAR_YEAR | CAL_MONTH_NO | CAL_MONTH | CAL_DAY_IN_MONTH | CAL_DAY_IN_YEAR | CAL_DAY_IN_WEEK |
|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 01-JAN-53 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | 20190102 | 02-JAN-19 | 2019 | 1 | 201901 | 2 | 2 | WED |
| 3 | 20190103 | 03-JAN-19 | 2019 | 1 | 201901 | 3 | 3 | THU |
| 4 | 20190105 | 05-JAN-19 | 2019 | 1 | 201901 | 5 | 5 | SAT |
| 5 | 20190107 | 07-JAN-19 | 2019 | 1 | 201901 | 7 | 7 | MON |

# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

**Type 1: Overwrite with current value:**

Type 1 SCD attributes are overwritten with the current value during the ETL refresh. So, only the current status of the values are reflected. For this model, the customer dimension table is a type1. We don't really care what our customer's past email addresses, and other core attributes where, only the current. Run a simple query against the customer_dim table and notice each customer only has one record, reflecting their current attributes.




| | CUSTOMER_KEY | CUSTOMER_ID | CUSTOMER_FIRST_NAME | CUSTOMER_LAST_NAME | CUSTOMER_PHONE | CURRENT_DEMO_KEY |
|---|---|---|---|---|---|---|
| 1 | 0 | 0 | N/A | N/A | N/A | 0 |
| 2 | 1 | 123456 | Albert | Einstein | 555-555-5555 | 70 |
| 3 | 2 | 123457 | Blaise | Pascal | 555-555-5555 | 25 |
| 4 | 3 | 123458 | Caroline | Herschel | 555-555-5555 | 14 |
| 5 | 4 | 123459 | Dorothy | Hodgkin | 555-555-5555 | 26 |
| 6 | 5 | 123460 | Edmond | Halley | 555-555-5555 | 47 |
| 7 | 6 | 1234561 | Elizabeth | Blackburn | 555-555-5555 | 49 |
| 8 | 7 | 1234562 | Erwin | Schroedinger | 555-555-5555 | 0 |
| 9 | 8 | 1234563 | Flossie | Wong-Staal | 555-555-5555 | 15 |
| 10 | 9 | 1234564 | Geraldine | Seydoux | 555-555-5555 | 26 |
| 11 | 10 | 1234565 | Ingrid | Daubechies | 555-555-5555 | 40 |
| 12 | 11 | 1234566 | Jane | Goodall | 555-555-5555 | 19 |
| 13 | 12 | 1234567 | Max | Born | 555-555-5555 | 2 |

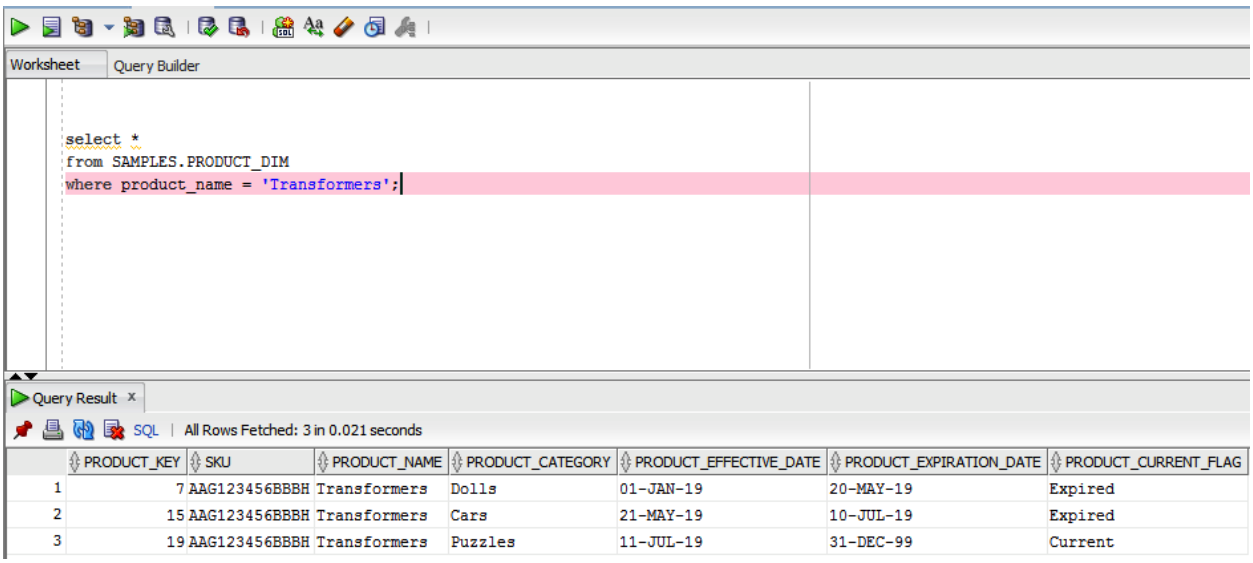
# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

**Type 2: Add a record when an attribute value changes:**

When an attribute history is tracked via type 2 SCD, a new record is added to the table each time the tracked attribute value changes. Our product dimension uses type 2 SCD to track the product category. Now run a query against the product dimension and filter on product_name = 'Transformers'.

```
select *
from SAMPLES.PRODUCT_DIM
where product_name = 'Transformers';
```




| | PRODUCT_KEY | SKU | PRODUCT_NAME | PRODUCT_CATEGORY | PRODUCT_EFFECTIVE_DATE | PRODUCT_EXPIRATION_DATE | PRODUCT_CURRENT_FLAG |
|---|---|---|---|---|---|---|---|
| 1 | 7 | AAG123456BBBH | Transformers | Dolls | 01-JAN-19 | 20-MAY-19 | Expired |
| 2 | 15 | AAG123456BBBH | Transformers | Cars | 21-MAY-19 | 10-JUL-19 | Expired |
| 3 | 19 | AAG123456BBBH | Transformers | Puzzles | 11-JUL-19 | 31-DEC-99 | Current |

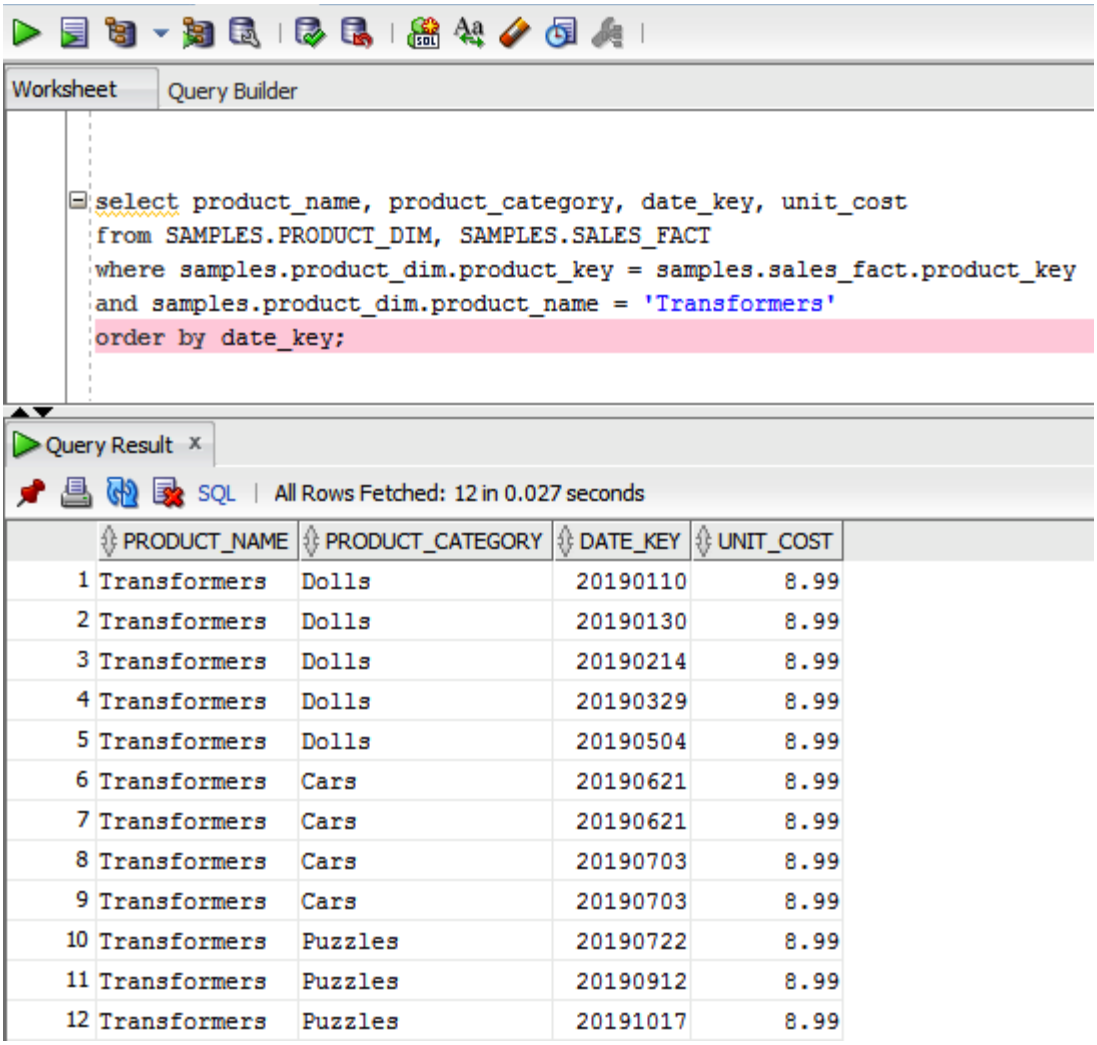
Notice there are three records related to the Transformers toy. Each time the Transformer toy's category changed, a new record was created. You can see the date ranges the category was in effect. You can also identify the current record by the product_current_flag attribute.

# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

;Now try another query, this time joining the product_dim to the sales_fact and filtering on product_name of ‘Transformers’.

```
select product_name, product_category, date_key, unit_cost
from SAMPLES.PRODUCT_DIM pd, SAMPLES.SALES_FACT sf
where pd.PRODUCT_KEY = sf.PRODUCT_KEY
and product_name = 'Transformers'
order by sf.DATE_KEY;
```




| | PRODUCT_NAME | PRODUCT_CATEGORY | DATE_KEY | UNIT_COST |
|---|---|---|---|---|
| 1 | Transformers | Dolls | 20190110 | 8.99 |
| 2 | Transformers | Dolls | 20190130 | 8.99 |
| 3 | Transformers | Dolls | 20190214 | 8.99 |
| 4 | Transformers | Dolls | 20190329 | 8.99 |
| 5 | Transformers | Dolls | 20190504 | 8.99 |
| 6 | Transformers | Cars | 20190621 | 8.99 |
| 7 | Transformers | Cars | 20190621 | 8.99 |
| 8 | Transformers | Cars | 20190703 | 8.99 |
| 9 | Transformers | Cars | 20190703 | 8.99 |
| 10 | Transformers | Puzzles | 20190722 | 8.99 |
| 11 | Transformers | Puzzles | 20190912 | 8.99 |
| 12 | Transformers | Puzzles | 20191017 | 8.99 |

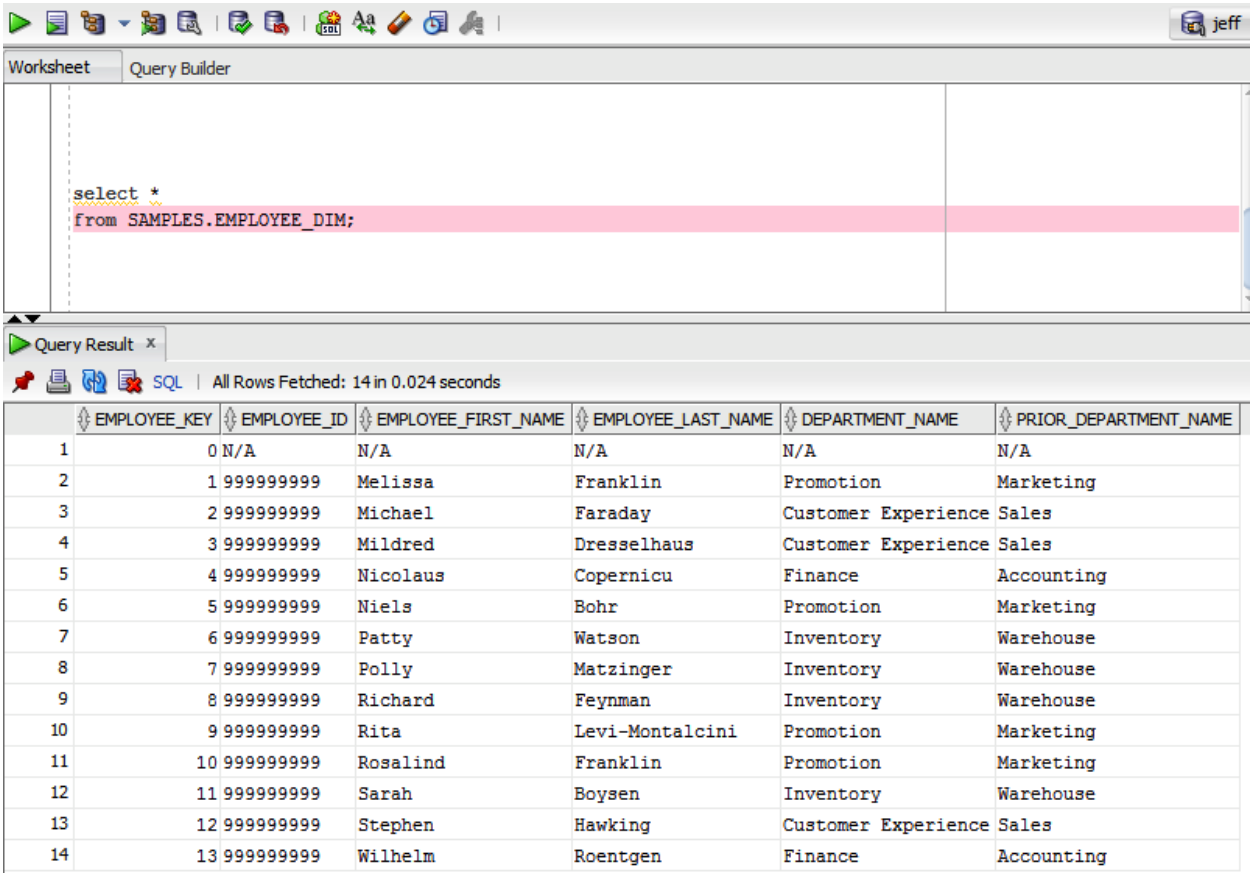
Notice that the correct product category is displayed for each transaction. The type 2 product_dim is giving you the correct historical reference for the product category at the time the transaction occurred.

# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

Type 3: Add a column to track the prior value:

A type 3 SCD allows you to track the current and prior value for an attribute by simply adding a prior value column. The clear limitation of Type 3 is that you can only track one change. This is often the approach for tracking a broad change, such as a large re-org. Run a simple query against the employee dimension. I decided to re-org my entire toy store, you will see the prior and current department for each of my employees.



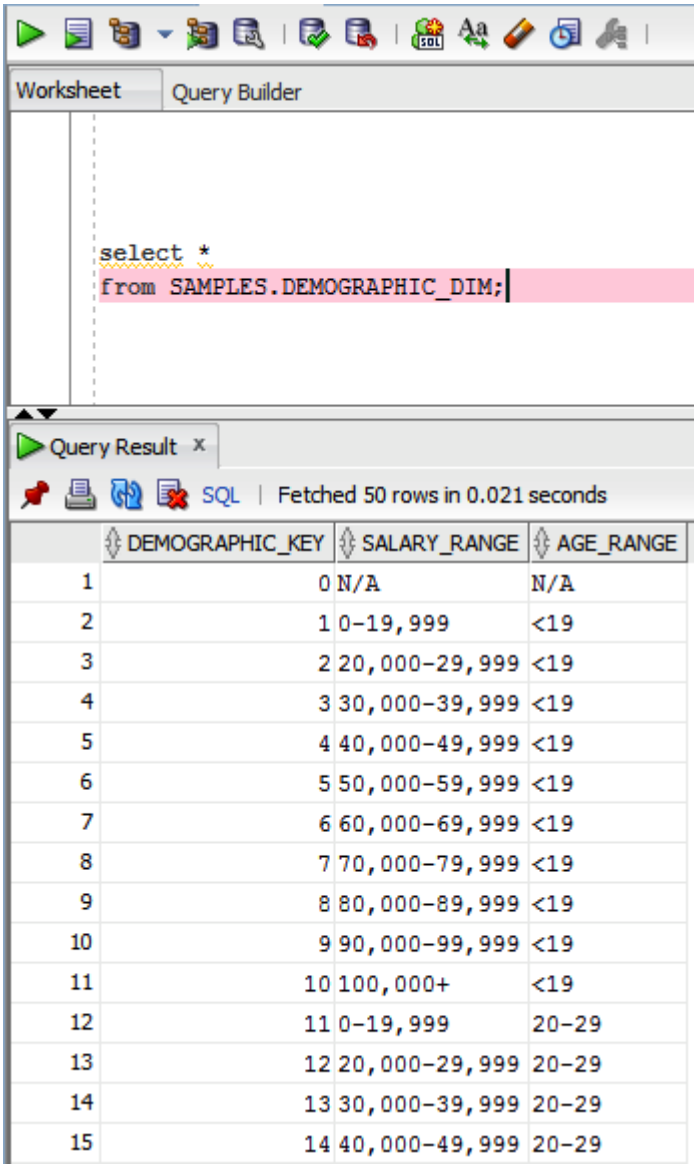

| | EMPLOYEE_KEY | EMPLOYEE_ID | EMPLOYEE_FIRST_NAME | EMPLOYEE_LAST_NAME | DEPARTMENT_NAME | PRIOR_DEPARTMENT_NAME |
|---|---|---|---|---|---|---|
| 1 | 0 | N/A | N/A | N/A | N/A | N/A |
| 2 | 1 | 999999999 | Melissa | Franklin | Promotion | Marketing |
| 3 | 2 | 999999999 | Michael | Faraday | Customer Experience | Sales |
| 4 | 3 | 999999999 | Mildred | Dresselhaus | Customer Experience | Sales |
| 5 | 4 | 999999999 | Nicolaus | Copernicu | Finance | Accounting |
| 6 | 5 | 999999999 | Niels | Bohr | Promotion | Marketing |
| 7 | 6 | 999999999 | Patty | Watson | Inventory | Warehouse |
| 8 | 7 | 999999999 | Polly | Matzinger | Inventory | Warehouse |
| 9 | 8 | 999999999 | Richard | Feynman | Inventory | Warehouse |
| 10 | 9 | 999999999 | Rita | Levi-Montalcini | Promotion | Marketing |
| 11 | 10 | 999999999 | Rosalind | Franklin | Promotion | Marketing |
| 12 | 11 | 999999999 | Sarah | Boysen | Inventory | Warehouse |
| 13 | 12 | 999999999 | Stephen | Hawking | Customer Experience | Sales |
| 14 | 13 | 999999999 | Wilhelm | Roentgen | Finance | Accounting |

# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

**Type 4: Add a mini dimension:**

Type 4 SCDs are good for tracking values with few distinct possible values and change frequently. For example customer demographics change frequently. If you track these frequently changing attributes via type 2, your customer dimension will get very big. Run a simple query against the demographic_dim table.



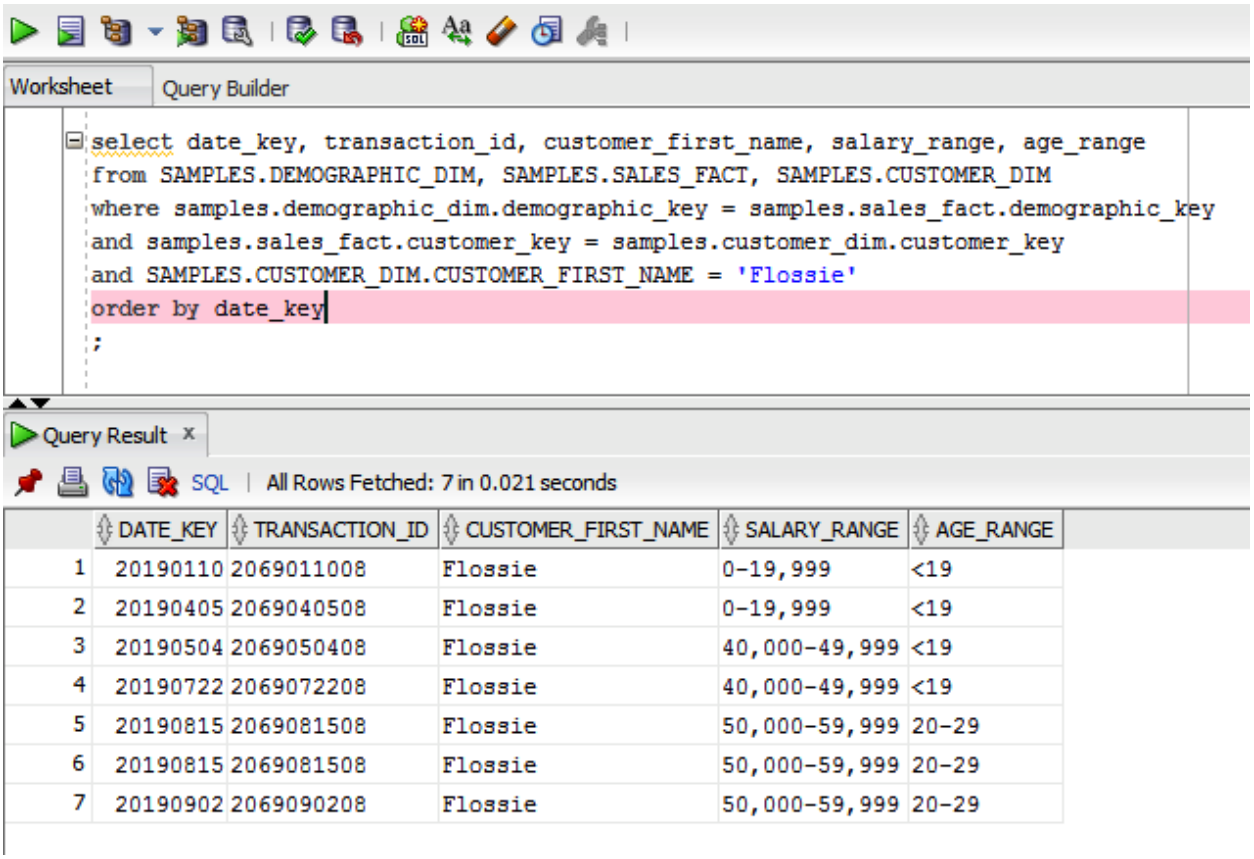

| | DEMOGRAPHIC_KEY | SALARY_RANGE | AGE_RANGE |
|---|---|---|---|
| 1 | 0 | N/A | N/A |
| 2 | 1 | 0-19,999 | <19 |
| 3 | 2 | 20,000-29,999 | <19 |
| 4 | 3 | 30,000-39,999 | <19 |
| 5 | 4 | 40,000-49,999 | <19 |
| 6 | 5 | 50,000-59,999 | <19 |
| 7 | 6 | 60,000-69,999 | <19 |
| 8 | 7 | 70,000-79,999 | <19 |
| 9 | 8 | 80,000-89,999 | <19 |
| 10 | 9 | 90,000-99,999 | <19 |
| 11 | 10 | 100,000+ | <19 |
| 12 | 11 | 0-19,999 | 20-29 |
| 13 | 12 | 20,000-29,999 | 20-29 |
| 14 | 13 | 30,000-39,999 | 20-29 |
| 15 | 14 | 40,000-49,999 | 20-29 |

# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

A Type 4 SCD will contain every possible combination of the attribute values. In this case the values are customer demographic attributes: salary range and age range. These attributes will definitely change over time, so we can easily track this change by associating the appropriate record to the sales fact when a purchase is made. Run a query, joining the demographic_dim and customer_dim to the sales_fact, and add a filter on customer_first_name = 'Flossie'.

```
select date_key,transaction_id, customer_first_name, salary_range, age_range
from SAMPLES.DEMOGRAPHIC_DIM dd, SAMPLES.SALES_FACT sf, SAMPLES.CUSTOMER_DIM cd
where dd.DEMOGRAPHIC_KEY = sf.DEMOGRAPHIC_KEY
and sf.CUSTOMER_KEY = cd.CUSTOMER_KEY
and cd.CUSTOMER_FIRST_NAME = 'Flossie'
order by date_key;
```




| | DATE_KEY | TRANSACTION_ID | CUSTOMER_FIRST_NAME | SALARY_RANGE | AGE_RANGE |
|---|---|---|---|---|---|
| 1 | 20190110 | 2069011008 | Flossie | 0-19,999 | <19 |
| 2 | 20190405 | 2069040508 | Flossie | 0-19,999 | <19 |
| 3 | 20190504 | 2069050408 | Flossie | 40,000-49,999 | <19 |
| 4 | 20190722 | 2069072208 | Flossie | 40,000-49,999 | <19 |
| 5 | 20190815 | 2069081508 | Flossie | 50,000-59,999 | 20-29 |
| 6 | 20190815 | 2069081508 | Flossie | 50,000-59,999 | 20-29 |
| 7 | 20190902 | 2069090208 | Flossie | 50,000-59,999 | 20-29 |

Notice our customer Flossie has changed salary range and age range during the time she has been shopping at our toy store. We can tell what her demographic profile was at the time of each purchase.

# Slowly Changing Dimensions – SQL Lab

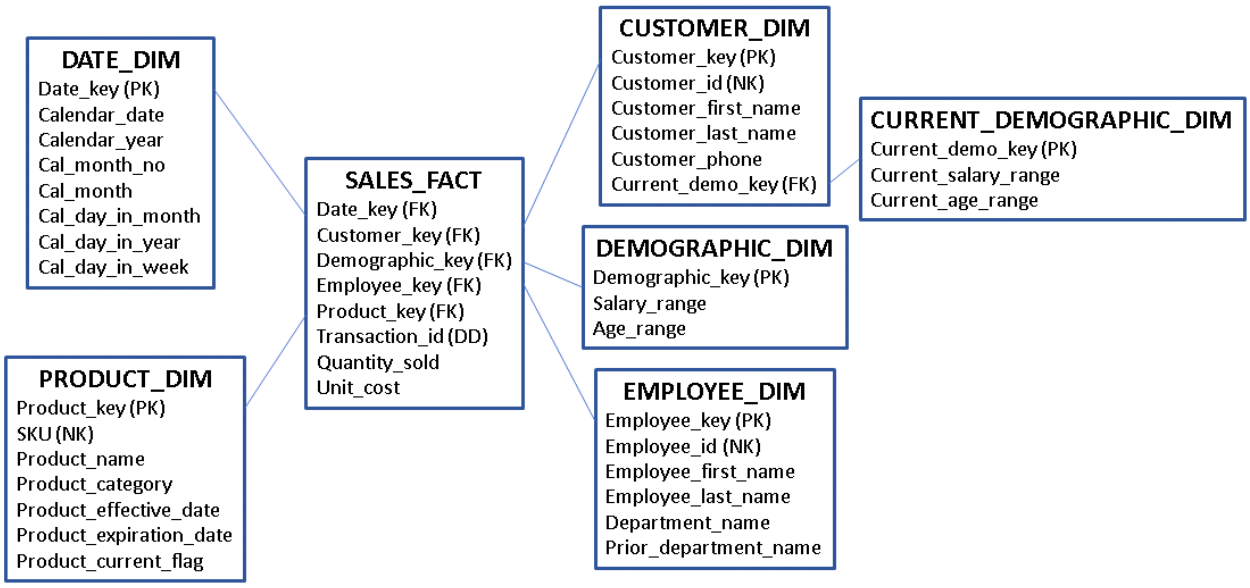
INFO4555 Business Intelligence Systems

**Type 5 SCD: Mini-dimension and Type 1 Outrigger:**

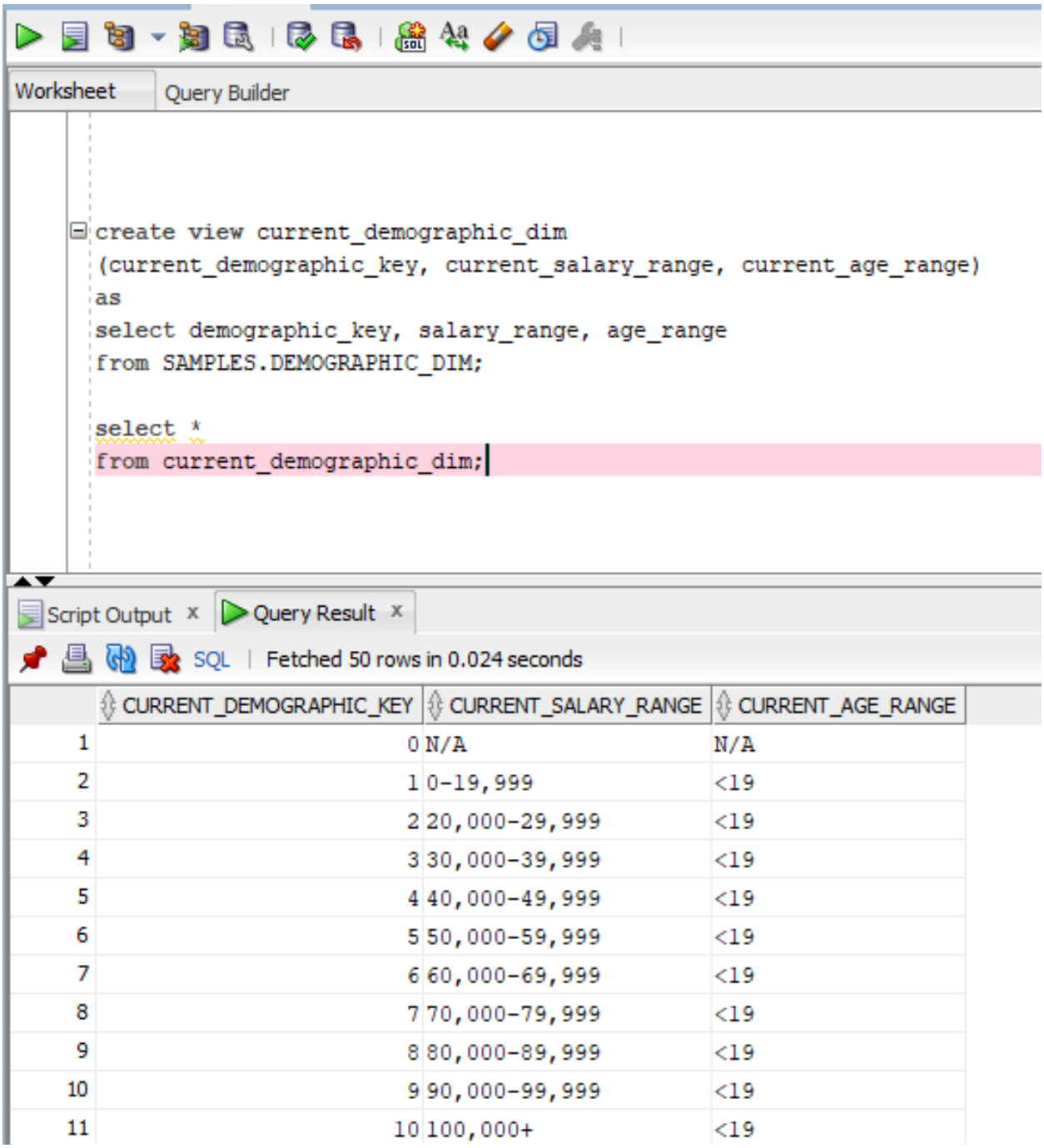
A Type 5 SCD is good when you have a frequently changing attribute and you need to track both the historical and current values for attributes. First, we create a role play view, over the mini-dimension. You will create roll play view called current_demographic_dim which references the table demographic_dim. Use the following SQL to create this roll play view in your own account:

```
create view current_demographic_dim
(current_demographic_key, current_salary_range, current_age_range)
as
select demographic_key, salary_range, age_range
from SAMPLES.DEMOGRAPHIC_DIM;
```

**Toy Store Retail Sales Dimensional Data Model**



Now select from the role play view you just created. Notice that you do not include the schema qualification (samples). This is because the view was created in your schema, not the samples schema.



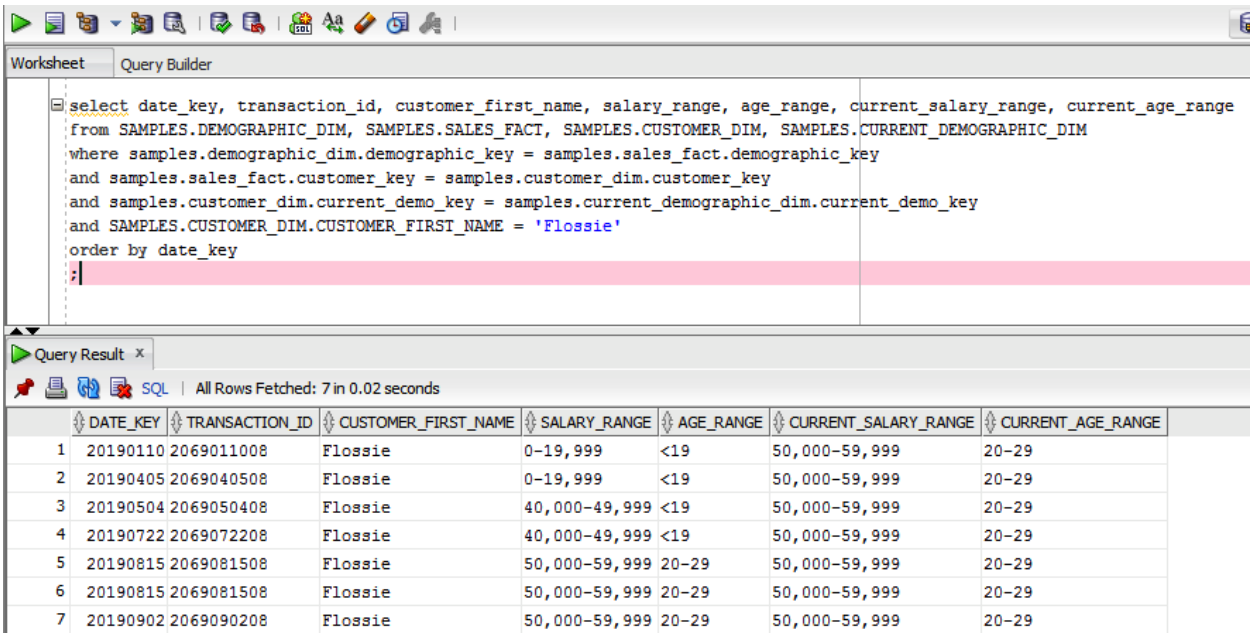

| | CURRENT_DEMOGRAPHIC_KEY | CURRENT_SALARY_RANGE | CURRENT_AGE_RANGE |
|---|---|---|---|
| 1 | 0 | N/A | N/A |
| 2 | 1 | 0-19,999 | <19 |
| 3 | 2 | 20,000-29,999 | <19 |
| 4 | 3 | 30,000-39,999 | <19 |
| 5 | 4 | 40,000-49,999 | <19 |
| 6 | 5 | 50,000-59,999 | <19 |
| 7 | 6 | 60,000-69,999 | <19 |
| 8 | 7 | 70,000-79,999 | <19 |
| 9 | 8 | 80,000-89,999 | <19 |
| 10 | 9 | 90,000-99,999 | <19 |
| 11 | 10 | 100,000+ | <19 |

Notice the data is from the underlying table demographic_dim, but we have given it a new name and column names via the view. Now join the current_demographic_dim to the customer_dim.

# Slowly Changing Dimensions – SQL Lab

INFO4555 Business Intelligence Systems

```
select date_key, transaction_id, customer_first_name, salary_range, age_range, current_salary_range,
current_age_range
from SAMPLES.DEMOGRAPHIC_DIM dd, SAMPLES.SALES_FACT sf, SAMPLES.CUSTOMER_DIM cd,
CURRENT_DEMOGRAPHIC_DIM cdd
where dd.DEMOGRAPHIC_KEY =  sf.DEMOGRAPHIC_KEY
and sf.CUSTOMER_KEY = cd.CUSTOMER_KEY
and cd.CURRENT_DEMO_KEY = cdd.CURRENT_DEMOGRAPHIC_KEY
and cd.CUSTOMER_FIRST_NAME = 'Flossie'
order by date_key;
```



Worksheet | Query Builder

```
select date_key, transaction_id, customer_first_name, salary_range, age_range, current_salary_range, current_age_range
from SAMPLES.DEMOGRAPHIC_DIM, SAMPLES.SALES_FACT, SAMPLES.CUSTOMER_DIM, SAMPLES.CURRENT_DEMOGRAPHIC_DIM
where samples.demographic_dim.demographic_key = samples.sales_fact.demographic_key
and samples.sales_fact.customer_key = samples.customer_dim.customer_key
and samples.customer_dim.current_demo_key = samples.current_demographic_dim.current_demo_key
and SAMPLES.CUSTOMER_DIM.CUSTOMER_FIRST_NAME = 'Flossie'
order by date_key
;
```

Query Result | SQL | All Rows Fetched: 7 in 0.02 seconds

| | DATE_KEY | TRANSACTION_ID | CUSTOMER_FIRST_NAME | SALARY_RANGE | AGE_RANGE | CURRENT_SALARY_RANGE | CURRENT_AGE_RANGE |
|---|---|---|---|---|---|---|---|
| 1 | 20190110 | 2069011008 | Flossie | 0-19,999 | <19 | 50,000-59,999 | 20-29 |
| 2 | 20190405 | 2069040508 | Flossie | 0-19,999 | <19 | 50,000-59,999 | 20-29 |
| 3 | 20190504 | 2069050408 | Flossie | 40,000-49,999 | <19 | 50,000-59,999 | 20-29 |
| 4 | 20190722 | 2069072208 | Flossie | 40,000-49,999 | <19 | 50,000-59,999 | 20-29 |
| 5 | 20190815 | 2069081508 | Flossie | 50,000-59,999 | 20-29 | 50,000-59,999 | 20-29 |
| 6 | 20190815 | 2069081508 | Flossie | 50,000-59,999 | 20-29 | 50,000-59,999 | 20-29 |
| 7 | 20190902 | 2069090208 | Flossie | 50,000-59,999 | 20-29 | 50,000-59,999 | 20-29 |

Now you can see both the current demographic profile, as well as, the historic.